Quant II

Lab 4

Yinxuan Wang

2026-02-26

Today's Agenda

- Effective Sample Size
- Weighting
- Specification Error
- Double Machine Learning
- AIPW

Effective Samples Intuition

- Consider a unit whose D is perfectly explained by X
- Then, does this unit help identify the effect of D|X on Y?

Effective Samples Intuition

- Simulation
- True treatment effect is 1
- How is X related to D here?

```r
set.seed(12)
N <- 1000
X <- rnorm(N)
D <- X + rbinom(N, size = 1, prob = 0.10) * rnorm(N)
Y <- D + X + rnorm(N)
df <- data.frame(X = X, D = D, Y = Y)
```

Effective Samples Intuition

- Simulation
- True treatment effect is 1
- How is X related to D here?

```
set.seed(12)
N <- 1000
X <- rnorm(N)
D <- X + rbinom(N, size = 1, prob = 0.10) * rnorm(N)
Y <- D + X + rnorm(N)
df <- data.frame(X = X, D = D, Y = Y)
```

```
table(df$X == df$D)
```

```
FALSE  TRUE
  113   887
```

Effective Samples: Intuition

```r
models <- list(
    feols(Y ~ D + X, data = df),
    feols(Y ~ D + X, data = df %>% filter(df$X != df$D))
)
```

```
                            model 1             model 2
Dependent Var.:                   Y                   Y

Constant              -0.0192 (0.0336)    0.0570 (0.0961)
D                    1.019*** (0.0936)  1.015*** (0.0901)
X                   0.9519*** (0.1022) 0.9995*** (0.1384)

---------------- ------------------- -------------------
S.E. type                        IID                 IID
Observations                   1,000                 113
---
Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Effective Sample Weights

- Observations where D is already "explained" by X are almost useless

Effective Sample Weights

- Observations where D is already "explained" by X are almost useless
- Effective Sample Weights formalize this intuition

Effective Sample Weights

- Observations where D is already "explained" by X are almost useless
- Effective Sample Weights formalize this intuition
- The effective sample weight for unit i is $\hat{w}_i = (D_i - \hat{E}[D_i|X_i])^2$

Effective Sample Weights

- Observations where D is already "explained" by X are almost useless
- Effective Sample Weights formalize this intuition
- The effective sample weight for unit i is $\hat{w}_i = (D_i - \hat{E}[D_i|X_i])^2$
  - If we assume linearity of the treatment assignment in X_i, then

Effective Sample Weights

- Observations where D is already "explained" by X are almost useless
- Effective Sample Weights formalize this intuition
- The effective sample weight for unit i is $\hat{w}_i = (D_i - \hat{E}[D_i|X_i])^2$
  - If we assume linearity of the treatment assignment in X_i, then
  - Run the regression $D_i = X_i\gamma + e_i$

Effective Sample Weights

- Observations where D is already "explained" by X are almost useless
- Effective Sample Weights formalize this intuition
- The effective sample weight for unit i is $\hat{w}_i = (D_i - \hat{E}[D_i|X_i])^2$
  - If we assume linearity of the treatment assignment in X_i, then
  - Run the regression $D_i = X_i\gamma + e_i$
  - Take residual $\hat{e}_i = D_i - X_i\hat{\gamma}$ and square it

What can you do with the sample weights?

- Use them to characterize your "effective" sample

What can you do with the sample weights?

- Use them to characterize your "effective" sample
- Maybe some substantive interpretation

What can you do with the sample weights?

- Use them to characterize your "effective" sample
- Maybe some substantive interpretation
- Eg, original sample is "representative," but is the effective sample "representative?"

What can you do with the sample weights?

- Use them to characterize your "effective" sample
- Maybe some substantive interpretation
- Eg, original sample is "representative," but is the effective sample "representative?"
- Interpretation depends on the research setting. Controlling for confounders likely makes the effective sample non representative, if you started with a representative sample.

Example: Egan/Mullin 2012

Turning Personal Experience into Political Attitudes: The Effect of Local Weather on Americans' Perceptions about Global Warming

Patrick J. Egan New York University
Megan Mullin Temple University

How do people translate their personal experiences into political attitudes? It has been difficult to explore this question using observational data, because individuals are typically exposed to experiences in a selective fashion, and self-reports of exposure may be biased and unreliable. In this study, we identify one experience to which Americans are exposed nearly at random—their local weather—and show that weather patterns have a significant effect on people's beliefs about the evidence for global warming.

Example: Egan/Mullin 2012

- Outcome variable `getwarmord` (attitudes about climate change)
  - "From what you've read and heard, is there solid evidence that the average temperature on earth has been getting warmer over the past few decades, or not?"

Example: Egan/Mullin 2012

- Outcome variable `getwarmord` (attitudes about climate change)
  - "From what you've read and heard, is there solid evidence that the average temperature on earth has been getting warmer over the past few decades, or not?"
- Treatment variable `ddtweek` (departure from normal daily average local temperature)

Example: Egan/Mullin 2012

- Outcome variable `getwarmord` (attitudes about climate change)
  - "From what you've read and heard, is there solid evidence that the average temperature on earth has been getting warmer over the past few decades, or not?"
- Treatment variable `ddtweek` (departure from normal daily average local temperature)

```
out.d <- feols(ddt_week ~ educ_hsless + educ_coll + educ_postgrad +
    educ_dk + party_rep + party_leanrep + party_leandem +
    party_dem + male + raceeth_black + raceeth_hisp +
    raceeth_notwbh + raceeth_dkref + age_1824 + age_2534 +
    age_3544 + age_5564 + age_65plus + age_dk + ideo_vcons +
    ideo_conservative + ideo_liberal + ideo_vlib + ideo_dk +
    attend_1 + attend_2 + attend_3 + attend_5 + attend_6 +
    attend_9 | doi + state + wbnid_num, d)
```

Example: Egan/Mullin 2012

- Outcome variable `getwarmord` (attitudes about climate change)
  - "From what you've read and heard, is there solid evidence that the average temperature on earth has been getting warmer over the past few decades, or not?"
- Treatment variable `ddtweek` (departure from normal daily average local temperature)

```
out.d <- feols(ddt_week ~ educ_hsless + educ_coll + educ_postgrad +
    educ_dk + party_rep + party_leanrep + party_leandem +
    party_dem + male + raceeth_black + raceeth_hisp +
    raceeth_notwbh + raceeth_dkref + age_1824 + age_2534 +
    age_3544 + age_5564 + age_65plus + age_dk + ideo_vcons +
    ideo_conservative + ideo_liberal + ideo_vlib + ideo_dk +
    attend_1 + attend_2 + attend_3 + attend_5 + attend_6 +
    attend_9 | doi + state + wbnid_num, d)
```

```
# Extract the residuals and take their square
d$wts <- residuals(out.d)^2
```

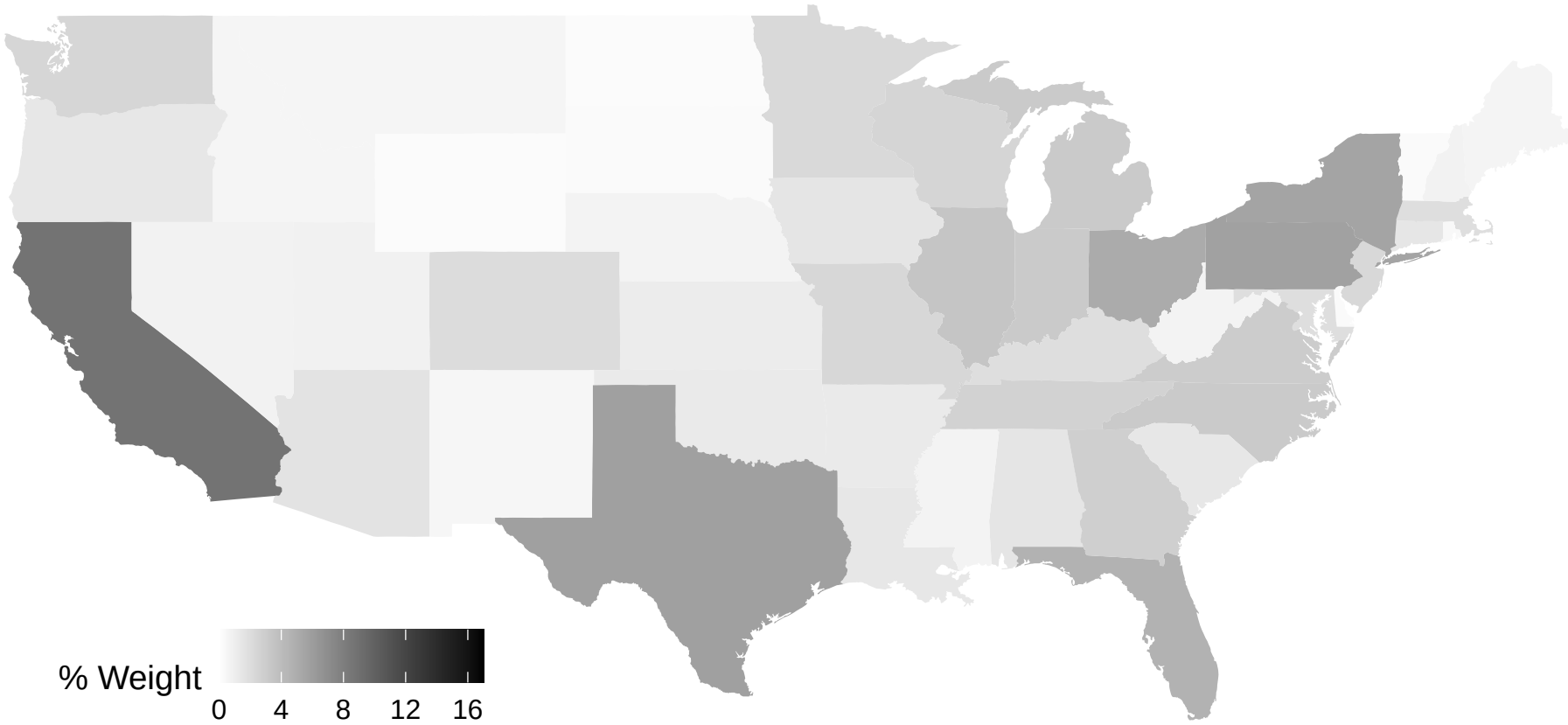
Example: Egan/Mullin 2012

- Suppose we want to characterize the sample contribution by state
- "Nominal" weight: just the (normalized) number of observations per state

```
nom_map <- theme_state_map(d %>%
    group_by(state) %>%
    summarize(nom = n() * 100 / nrow(d)) %>%
    ggplot(aes(map_id = state)) +
    geom_map(aes(fill = nom), map = state_map) +
    labs(title = "Nominal Sample"))
```

Example: Egan/Mullin 2012

Nominal Sample



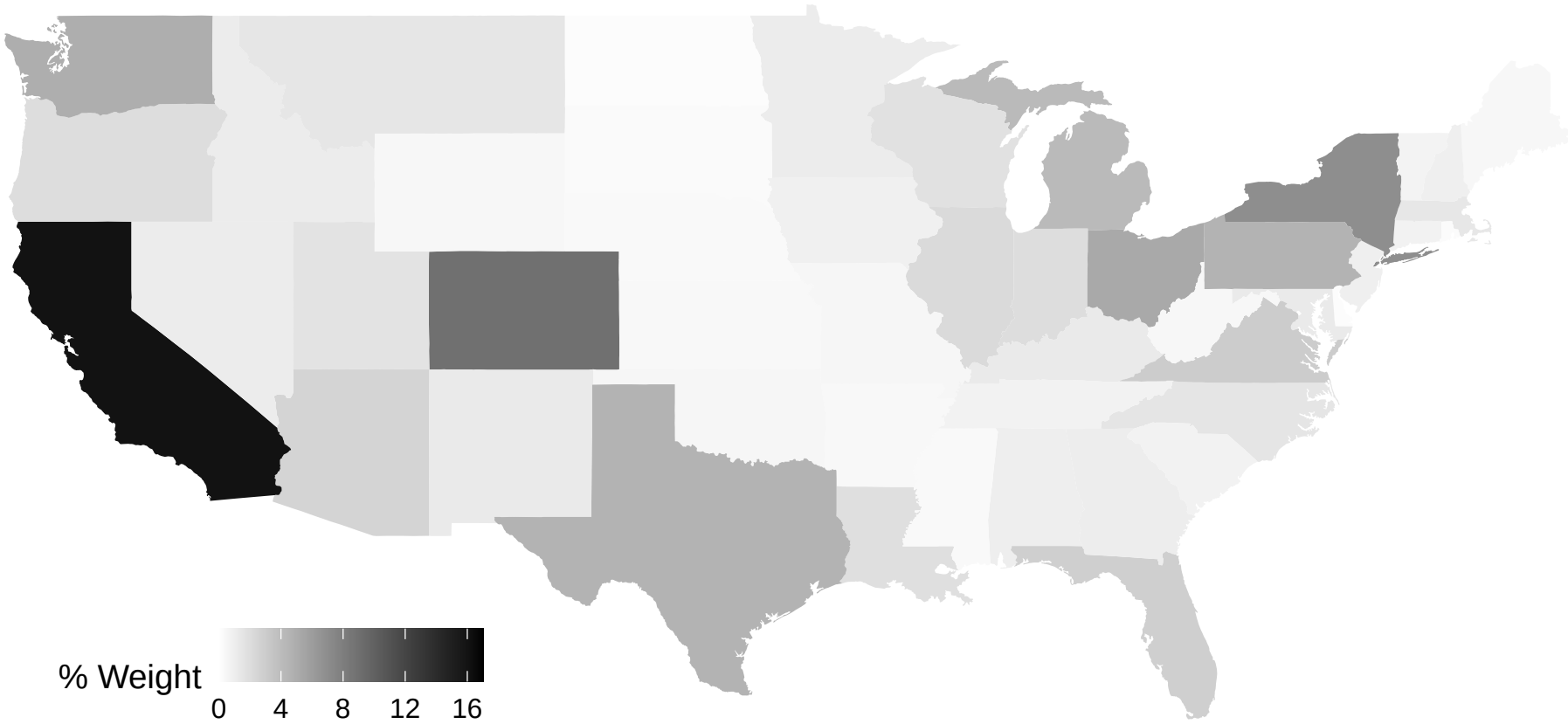
Example: Egan/Mullin 2012

- To characterize the "effective" contribution of each state, use the effective sample weight instead

```
eff_map <- theme_state_map(d %>%
    group_by(state) %>%
    summarize(eff = sum(wts) * 100 / sum(d$wts)) %>%
    ggplot(aes(map_id = state)) +
    geom_map(aes(fill = eff), map = state_map) +
    labs(title = "Effective Sample"))
```

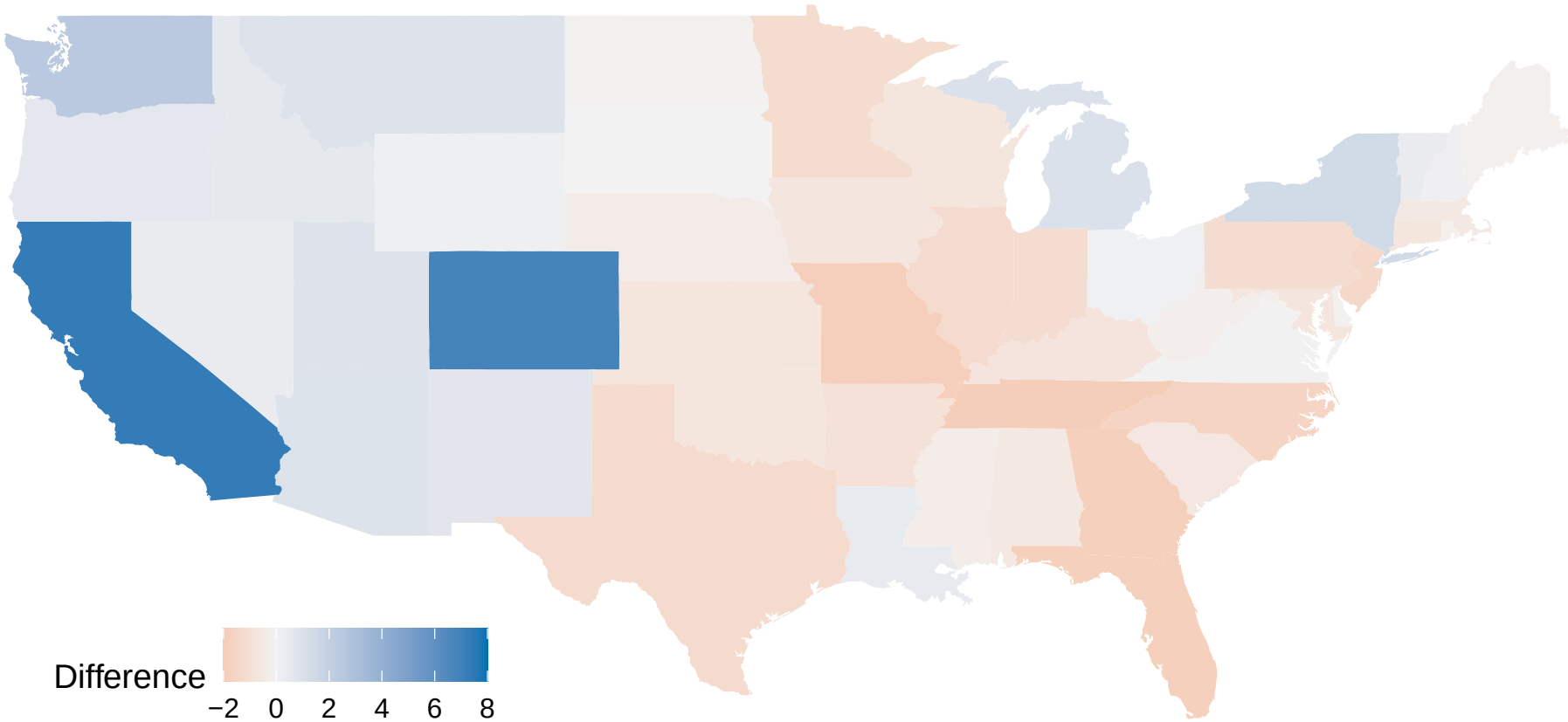
Example: Egan/Mullin 2012

Effective Sample



Example: Egan/Mullin 2012

Difference Effective and Nominal Weight



Example: Egan/Mullin 2012

- One possible critique here:
  - Coastal cities in WA, CA have extremely variable weather
  - Rural areas of WA, CA have extremely stable weather
  - We control for liberal/conservative, but maybe there's urban/rural confounding slightly separate driving results?

Propensity score

- The propensity score is defined as

$$e(X_i) = P(D_i = 1|X_i) = E(D_i|X_i)$$

- Under strong ignorability, we have

(Rosenbaum and Rubin)

$$D_i \perp (Y_{0i}, Y_{1i}) \mid e(X_i)$$

$$D_i \perp X_i \mid e(X_i)$$

Weighting

- Inverse probability weighting (IPW) is appealing
- With correct $e(X)$ and overlap, it identifies the PATE under CIA by creating a pseudo-population where D is independent of X

Weighting

- Inverse probability weighting (IPW) is appealing
- With correct $e(X)$ and overlap, it identifies the PATE under CIA by creating a pseudo-population where D is independent of X

$$
\begin{aligned}
E\left[\frac{D_i Y_i}{e(X_i)} - \frac{(1-D_i)Y_i}{1-e(X_i)}\right] &= E\left[\frac{D_i Y_{1i}}{e(X_i)} - \frac{(1-D_i)Y_{0i}}{1-e(X_i)}\right] \\
&= E\left[E\left[\frac{D_i Y_{1i}}{e(X_i)} - \frac{(1-D_i)Y_{0i}}{1-e(X_i)} \mid X_i\right]\right] \\
&= E\left[\frac{E[D_i|X_i]E[Y_{1i}|X_i]}{e(X_i)} - \frac{E[1-D_i|X_i]E[Y_{0i}|X_i]}{1-e(X_i)}\right] \\
&= E\left[\frac{e(X_i)E[Y_{1i}|X_i]}{e(X_i)} - \frac{(1-e(X_i))E[Y_{0i}|X_i]}{1-e(X_i)}\right] = \\
&= E[E[Y_{1i} - Y_{0i}|X_i]] = E[Y_{1i} - Y_{0i}] \\
&= \tau_{PATE}
\end{aligned}
$$

Weighting

- Inverse probability weighting (IPW) is appealing
- With correct $e(X)$ and overlap, it identifies the PATE under CIA by creating a pseudo-population where D is independent of X

$$
\begin{aligned}
E\left[\frac{D_i Y_i}{e(X_i)} - \frac{(1-D_i)Y_i}{1-e(X_i)}\right] &= E\left[\frac{D_i Y_{1i}}{e(X_i)} - \frac{(1-D_i)Y_{0i}}{1-e(X_i)}\right] \\
&= E\left[E\left[\frac{D_i Y_{1i}}{e(X_i)} - \frac{(1-D_i)Y_{0i}}{1-e(X_i)} \mid X_i\right]\right] \\
&= E\left[\frac{E[D_i|X_i]E[Y_{1i}|X_i]}{e(X_i)} - \frac{E[1-D_i|X_i]E[Y_{0i}|X_i]}{1-e(X_i)}\right] \\
&= E\left[\frac{e(X_i)E[Y_{1i}|X_i]}{e(X_i)} - \frac{(1-e(X_i))E[Y_{0i}|X_i]}{1-e(X_i)}\right] = \\
&= E[E[Y_{1i} - Y_{0i}|X_i]] = E[Y_{1i} - Y_{0i}] \\
&= \tau_{PATE}
\end{aligned}
$$

Unstable with extreme weights when $e(X_i)$ is near 0 or 1

Weighting

A natural weighting estimator is the IPW estimator.

$$\hat{\tau} = \frac{1}{N}\sum_{i=1}^{N}\frac{D_i Y_i}{\hat{e}(X_i)} - \frac{1}{N}\sum_{i=1}^{N}\frac{(1-D_i)Y_i}{1-\hat{e}(X_i)}$$

We usually use a "normalized" version of it.

$$\hat{\tau}_{ipw} = \frac{\sum_{i=1}^{N}\frac{D_i Y_i}{\hat{e}(X_i)}}{\sum_{i=1}^{N}\frac{D_i}{\hat{e}(X_i)}} - \frac{\sum_{i=1}^{N}\frac{(1-D_i)Y_i}{(1-\hat{e}(X_i))}}{\sum_{i=1}^{N}\frac{(1-D_i)}{(1-\hat{e}(X_i))}}$$

Shifts modeling burdens from estimating $E[Y_i|X_i]$ to estimating $e(X_i)$.

Weighting

- Other weighting estimators do not model the probability of treatment but target covariate balance directly

Weighting

- Other weighting estimators do not model the probability of treatment but target covariate balance directly
- E.g. Covariate Balancing Propensity Score, Entropy Balancing

Weighting

- Other weighting estimators do not model the probability of treatment but target covariate balance directly
- E.g. Covariate Balancing Propensity Score, Entropy Balancing
- In R: all available in the package `WeightIt`

Weighting

- Other weighting estimators do not model the probability of treatment but target covariate balance directly
- E.g. Covariate Balancing Propensity Score, Entropy Balancing
- In R: all available in the package `WeightIt`
  - Other packages: `CBPS`, `PSweight`, `Causalweight`

Weighting

- Other weighting estimators do not model the probability of treatment but target covariate balance directly
- E.g. Covariate Balancing Propensity Score, Entropy Balancing
- In R: all available in the package `WeightIt`
  - Other packages: `CBPS`, `PSweight`, `Causalweight`
- Methods allowed for IPW: glm (default), non-parametric methods, SuperLearner

Weighting

Application to climate change opinions example

```r
# Use WeightIt package
pacman::p_load(WeightIt)

# Binary indicator for treatment
d$treat <- ifelse(d$ddt_week>quantile(d$ddt_week, 0.75),1,0)

# Propensity score weighting
wd <- weightit(treat ~ educ_hsless + educ_coll + educ_postgrad +
    educ_dk + party_rep + party_leanrep + party_leandem +
    party_dem + male + raceeth_black + raceeth_hisp +
    raceeth_notwbh + raceeth_dkref + age_1824 + age_2534 +
    age_3544 + age_5564 + age_65plus + age_dk + ideo_vcons +
    ideo_conservative + ideo_liberal + ideo_vlib + ideo_dk +
    attend_1 + attend_2 + attend_3 + attend_5 + attend_6 +
    attend_9 + as.factor(statenum),
    data=d, estimand="ATT", method = "ps")

# Description
wd
```

```
A weightit object
 - method: "glm" (propensity score weighting with GLM)
 - number of obs.: 6726
 - sampling weights: none
 - treatment: 2-category
 - estimand: ATT (focal: 1)
 - covariates: educ_hsless, educ_coll, educ_postgrad, educ_dk, party_rep, party_leanrep, party_leandem, party_d
```

Weighting

```
# Describe the weights
summary(wd)
```

```
                 Summary of weights

- Weight ranges:

           Min                                  Max
treated 1.0000             ||                1.0000
control 0.0048 |---------------------------| 2.5009

- Units with the 5 most extreme weights by group:

             11    10      4      3      2
 treated      1     1      1      1      1
            164   208    199    191    179
 control 1.6262 1.797 1.8018 2.3558 2.5009

- Weight statistics:

        Coef of Var   MAD Entropy # Zeros
treated       0.000 0.000   0.000       0
control       0.936 0.781   0.436       0

- Effective Sample Sizes:

           Control Treated
Unweighted 5048.      1678
Weighted   2689.91    1678
```

Weighting

```
# Weighted difference in means
etable(feols(getwarmord ~ treat, data = d, weights = wd$weight
```

```
                  feols(getwarmor..
Dependent Var.:         getwarmord

Constant          2.535*** (0.0170)
treat              0.0495* (0.0239)
_________________ _________________
S.E.: Clustered       by: wbnid_num
Observations                  6,726
---
Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Specification Error

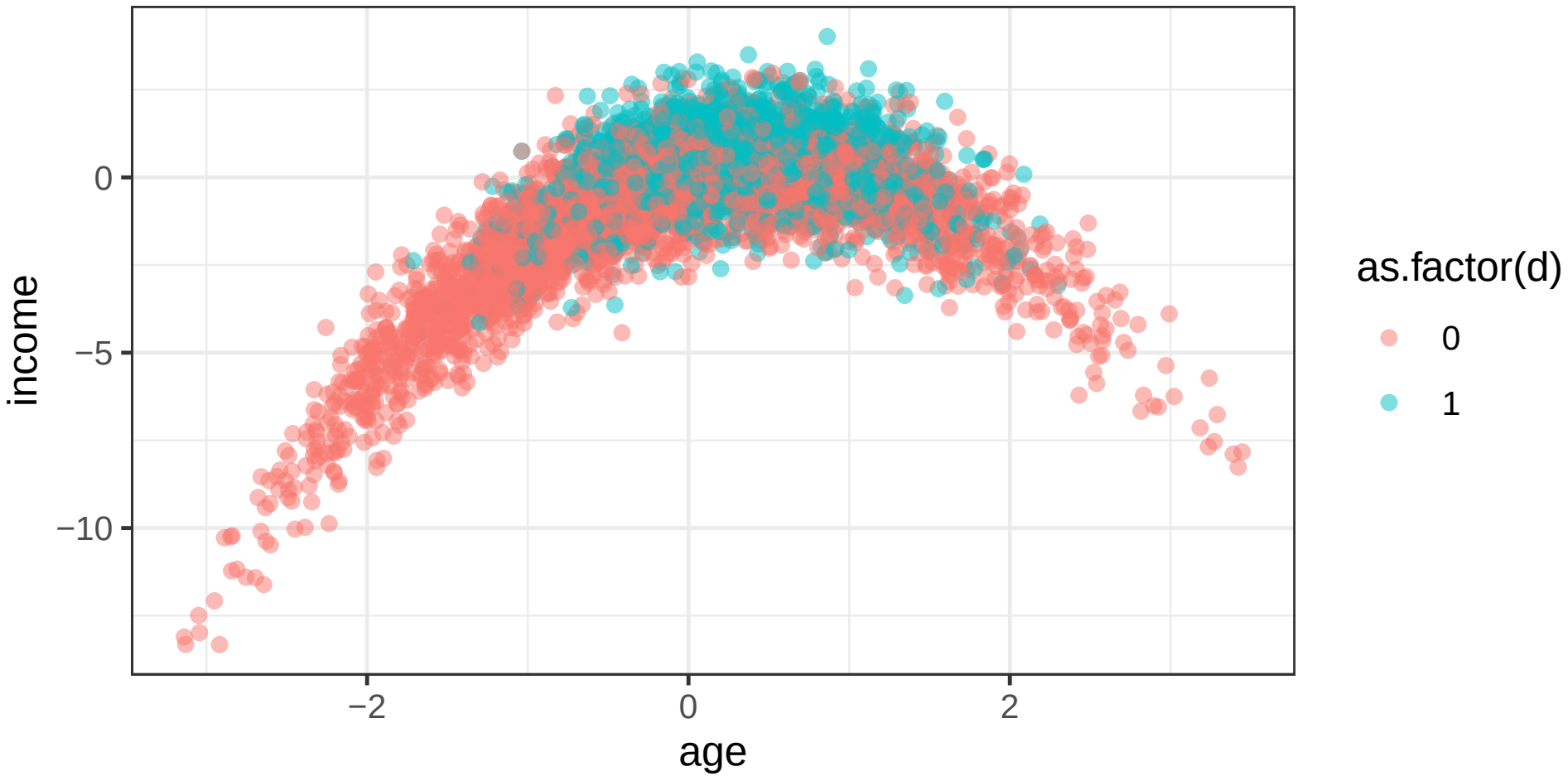
- Linear specification for controls is a substantive assumption
- Introduce bias when confounding is nonlinear and controls are misspecified
- Simulation:
  - True effect is 0.5
  - Nonlinearity in both y $\sim$ x and d $\sim$ x

Specification Error

- Linear specification for controls is a substantive assumption
- Introduce bias when confounding is nonlinear and controls are misspecified
- Simulation:
  - True effect is 0.5
  - Nonlinearity in both y ~ x and d ~ x

```
set.seed(123)
N <- 5000
effect <- 0.5
age <- rnorm(N, 0, 1)
age2 <- -(age)^2 + age
d <- rbinom(N, size = 1, prob = plogis(age2))
income <- -(age)^2 + age + rnorm(N) + d * effect
```

Specification Error



Specification Error

- Using a linear specification for control leads to bias in estimate

```
etable(feols(income ~ age + d, data = dat), fitstat = c("n"))
```

```
                  feols(income ~ a..
Dependent Var.:                 income

Constant           -1.297*** (0.0301)
age                0.8768*** (0.0245)
d                   1.335*** (0.0507)
------------------ ------------------
S.E. type                         IID
Observations                    5,000
---
Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Specification Error

- But, a problem quickly arises.

Specification Error

- But, a problem quickly arises.
- How do we know what specification to use?

Specification Error

- But, a problem quickly arises.
- How do we know what specification to use?
- "Classical" Machine Learning: flexible algorithms to estimate nonlinear relationships

Machine Learning Packages in R

- `mlr3`: unified interface for training, predicting, cross-validation, tuning hyperparameters; pipelines and benchmarking
- `mlr3learners`: companion package providing a catalog of model implementations ("learners") you can plug into `mlr3`
- `xgboost`: Gradient-boosted decision trees
- `ranger`: Random Forests

Machine Learning Packages in R

- `mlr3`: unified interface for training, predicting, cross-validation, tuning hyperparameters; pipelines and benchmarking
- `mlr3learners`: companion package providing a catalog of model implementations ("learners") you can plug into `mlr3`
- `xgboost`: Gradient-boosted decision trees
- `ranger`: Random Forests

If you're interested in ML

- Quant 3 - Machine Learning
- A Machine Learning Textbook:
  - Probabilistic Machine Learning: An Introduction by Kevin Patrick Murphy. MIT Press, March 2022.

Regularized Linear Regression: Ridge, Lasso, Elastic Net

Goal: Predict Y from many covariates X without overfitting by shrinking coefficients.

$$\hat{\beta} = \arg\min_{\beta} \frac{1}{N} \sum_{i=1}^{N} (y_i - x_i^\top \beta)^2 \; + \; \lambda \, \text{Penalty}(\beta)$$

- Lasso (L1): Penalty $= \sum_j |\beta_j|$
  - Shrinks and can set some coefficients exactly to 0.

Regularized Linear Regression: Ridge, Lasso, Elastic Net

Goal: Predict Y from many covariates X without overfitting by shrinking coefficients.

$$\hat{\beta} = \arg\min_{\beta} \frac{1}{N} \sum_{i=1}^{N} (y_i - x_i^\top \beta)^2 \; + \; \lambda \, \text{Penalty}(\beta)$$

- Lasso (L1): Penalty $= \sum_j |\beta_j|$
  - Shrinks and can set some coefficients exactly to 0.
- Ridge (L2): Penalty $= \sum_j \beta_j^2$
  - Shrinks coefficients toward 0; usually keeps all variables.

Regularized Linear Regression: Ridge, Lasso, Elastic Net

Goal: Predict Y from many covariates X without overfitting by shrinking coefficients.

$$\hat{\beta} = \arg\min_{\beta} \frac{1}{N} \sum_{i=1}^{N} (y_i - x_i^\top \beta)^2 \; + \; \lambda \, \text{Penalty}(\beta)$$

- Lasso (L1): Penalty $= \sum_j |\beta_j|$
  - Shrinks and can set some coefficients exactly to 0.
- Ridge (L2): Penalty $= \sum_j \beta_j^2$
  - Shrinks coefficients toward 0; usually keeps all variables.
- Elastic Net (L1 + L2): Penalty $= \alpha \sum_j |\beta_j| + (1-\alpha) \sum_j \beta_j^2$
- Choose λ (and α for Elastic Net) by cross-validation.

Cross Validation

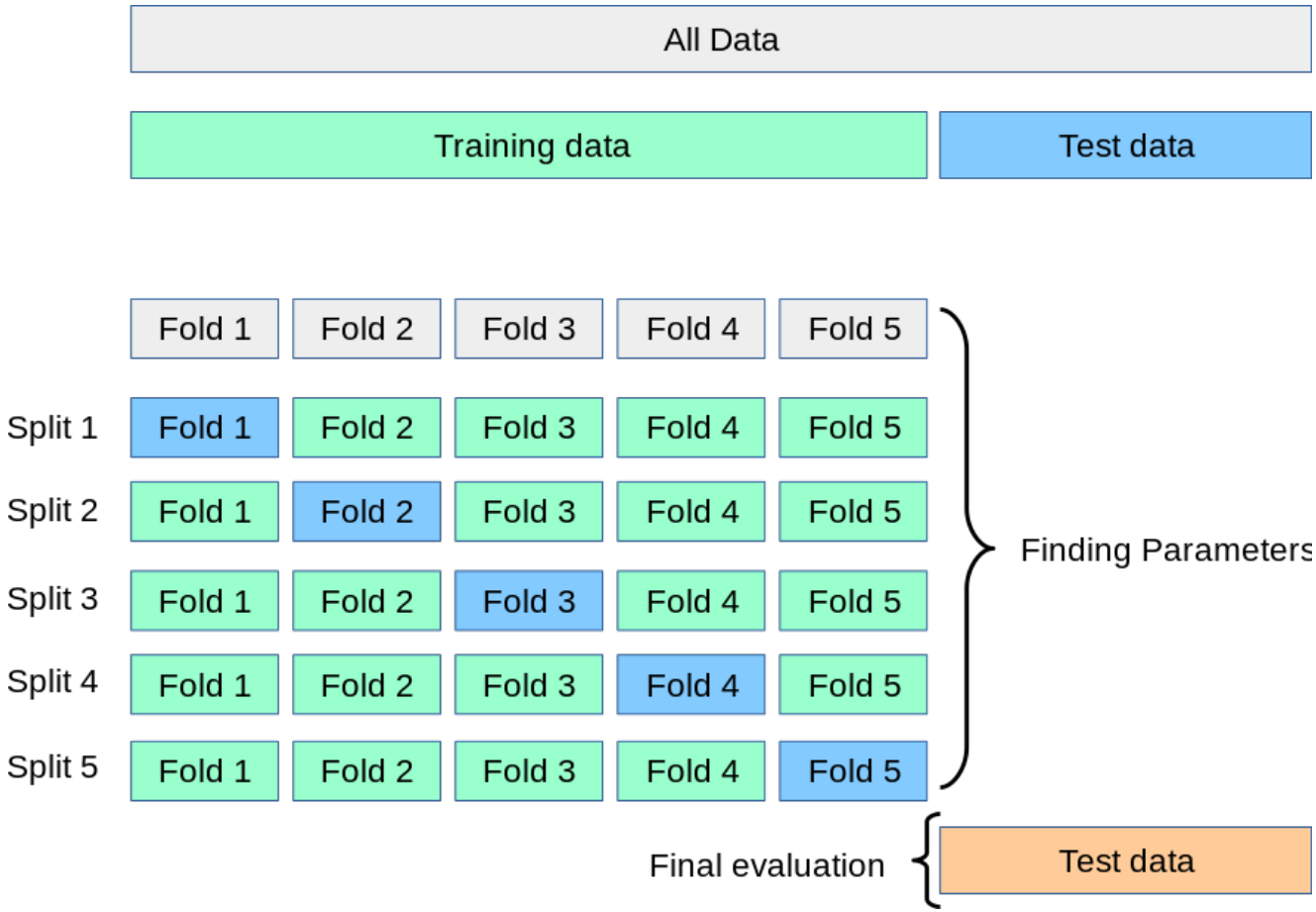
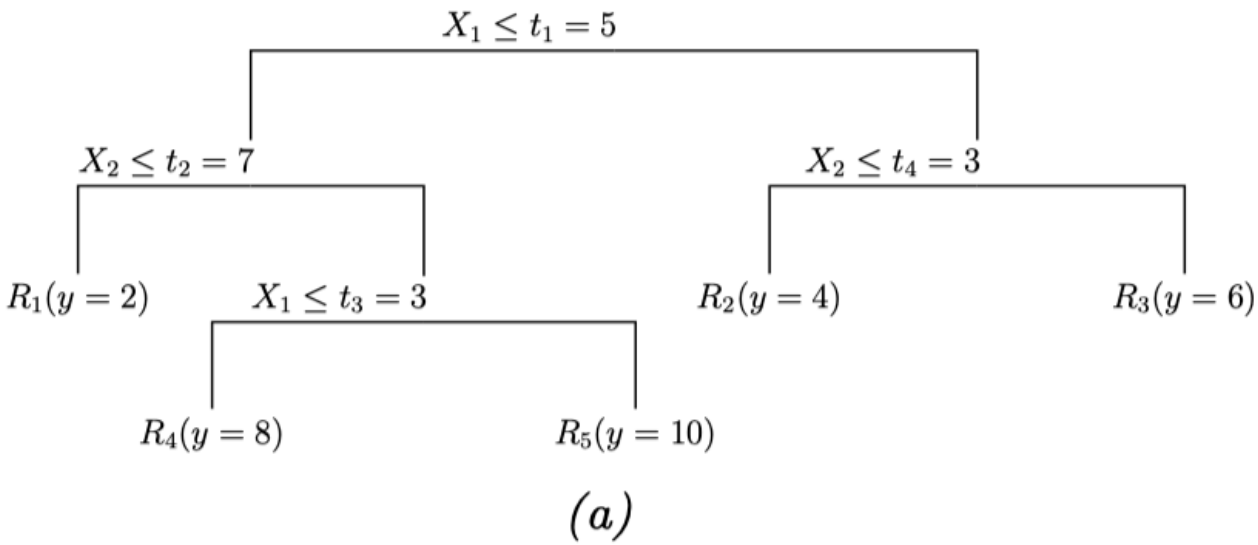


Image from sk-learn

Trees

Tree: a set of nested decision rules

- Recursively partitioning the input space
- Assign a local prediction to each region (a constant mean / class probability)



(Murphy 2022)

Trees

Training

- Minimize total prediction error on the training data.

Trees

Training

- Minimize total prediction error on the training data.
- Greedy procedure (CART)
  - iteratively grow the tree one node at a time
  - at each node, choose the split that gives the biggest immediate loss reduction
- Common hyperparameters: max depth, min leaf size, min split size, pruning/complexity, features tried per split.
- Tune by CV

Trees

Training

- Minimize total prediction error on the training data.
- Greedy procedure (CART)
  - iteratively grow the tree one node at a time
  - at each node, choose the split that gives the biggest immediate loss reduction
- Common hyperparameters: max depth, min leaf size, min split size, pruning/complexity, features tried per split.
- Tune by CV
- Prevent overfitting is critical - set maximum depth or *prune*

Trees

Training

- Minimize total prediction error on the training data.
- Greedy procedure (CART)
  - iteratively grow the tree one node at a time
  - at each node, choose the split that gives the biggest immediate loss reduction
- Common hyperparameters: max depth, min leaf size, min split size, pruning/complexity, features tried per split.
- Tune by CV
- Prevent overfitting is critical - set maximum depth or *prune*

Problem

- Easy to interpret but sensitive to small changes in the data
- High-variance problem

Trees

Training

- Minimize total prediction error on the training data.
- Greedy procedure (CART)
  - iteratively grow the tree one node at a time
  - at each node, choose the split that gives the biggest immediate loss reduction
- Common hyperparameters: max depth, min leaf size, min split size, pruning/complexity, features tried per split.
- Tune by CV
- Prevent overfitting is critical - set maximum depth or *prune*

Problem

- Easy to interpret but sensitive to small changes in the data
- High-variance problem

$\rightarrow$ Ensemble methods combining many trees

Bagging, Random Forest

Bagging (Bootstrap Aggregating)

- Problem: single trees have high variance (unstable predictions)
- Idea: fit many trees on bootstrap resamples, then aggregate predictions
  - averaging many noisy models reduces variance

Random Forests

- Bagging + extra randomness
- At each split, consider only a random subset of features (mtry)
  - Reduces correlation between trees, soaveraging becomes more effective

Boosting

- Build trees sequentially, each new tree improves on the current model
- Result is an additive model
- Intuition: each tree learns to predict the remaining errors
  - Each tree is scaled, usually by performance or a learning rate
- Easy to overfit
  - Regularization is crucial: shallow trees, early stopping, small learning rates
- Adaboost, gradient boosting, XGBoost (extreme gradient boosting)

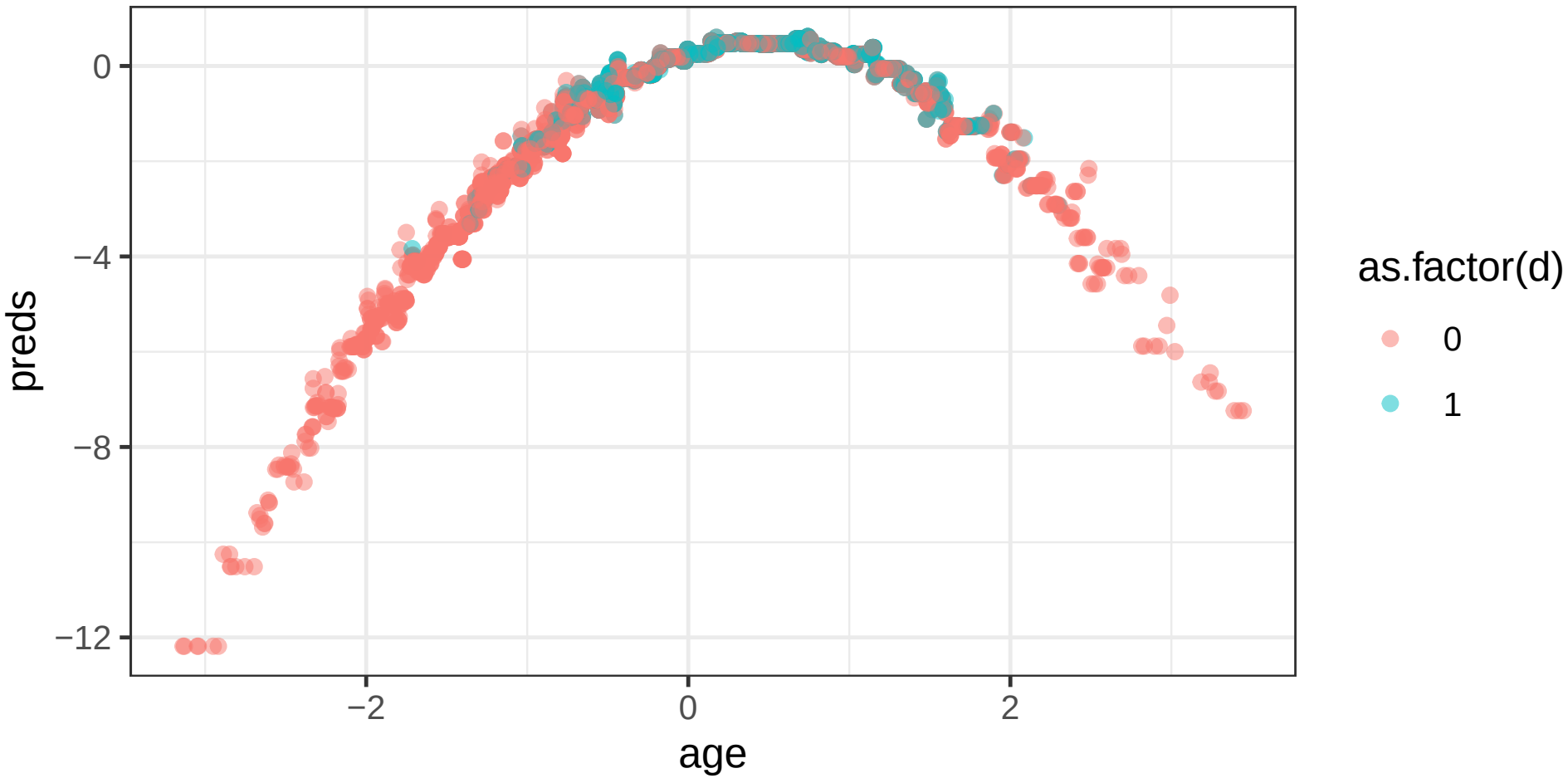
Classical Machine Learning

```
pacman::p_load(mlr3, xgboost, mlr3learners, ranger)

# Use XGBoost algorithm
learner <- lrn("regr.xgboost", eta = 0.1, nrounds = 35)
# Set up a task to predict 'income'
task <- as_task_regr(
    select(dat, income, age),
    target = "income"
)
# Fit to the data
learner$train(task)
```

Classical Machine Learning

```
dat$preds <- learner$predict_newdata(dat)$response
```



Double Machine Learning - partially linear regression

- Now, we can predict Y given X in a nonlinear way.

Double Machine Learning - partially linear regression

- Now, we can predict Y given X in a nonlinear way.
- How do we use this to retrieve a good estimate of `Y ~ D | X`?

Double Machine Learning - partially linear regression

- Now, we can predict Y given X in a nonlinear way.
- How do we use this to retrieve a good estimate of `Y ~ D | X`?
- Basic idea is to use ML to model both `y ~ x` and `d ~ x` and use the residuals to retrieve a consistent estimate for θ

PLR By Hand

```
# Model y as a function of age
y.x <- lrn("regr.xgboost", eta = 0.1, nrounds = 35)
y.x.task <- as_task_regr(
    select(dat, income, age),
    target = "income"
)
y.x$train(y.x.task)
```

PLR By Hand

```
# Model y as a function of age
y.x <- lrn("regr.xgboost", eta = 0.1, nrounds = 35)
y.x.task <- as_task_regr(
    select(dat, income, age),
    target = "income"
)
y.x$train(y.x.task)
```

```
# Model D as a function of age
d.x <- lrn("regr.xgboost", eta = 0.1, nrounds = 35)
d.x.task <- as_task_regr(
    select(dat, d, age),
    target = "d"
)
d.x$train(d.x.task)
```

PLR By Hand

```
# Model y as a function of age
y.x <- lrn("regr.xgboost", eta = 0.1, nrounds = 35)
y.x.task <- as_task_regr(
    select(dat, income, age),
    target = "income"
)
y.x$train(y.x.task)
```

```
# Model D as a function of age
d.x <- lrn("regr.xgboost", eta = 0.1, nrounds = 35)
d.x.task <- as_task_regr(
    select(dat, d, age),
    target = "d"
)
d.x$train(d.x.task)
```

```
# Calculate residuals
d.x.resid <- dat$d - d.x$predict_newdata(dat)$response
y.x.resid <- dat$income - y.x$predict_newdata(dat)$response
```

PLR By Hand

```r
# effect <- 0.5
# income <- -(age)^2 + age + rnorm(N) + d * effect
summary(lm(y.x.resid ~ d.x.resid))
```

```
Call:
lm(formula = y.x.resid ~ d.x.resid)

Residuals:
    Min      1Q  Median      3Q     Max
-3.9637 -0.6482 -0.0110  0.6651  3.8364

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) -0.03362    0.01360  -2.472   0.0135 *
d.x.resid    0.50320    0.03235  15.555   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.9618 on 4998 degrees of freedom
Multiple R-squared:  0.04618,   Adjusted R-squared:  0.04598
F-statistic:   242 on 1 and 4998 DF,  p-value: < 2.2e-16
```

DML PLR By Hand

```r
set.seed(123)

# dat must have columns: income (Y), d (D), age (X)
K <- 5
fold_id <- sample(rep(1:K, length.out = nrow(dat)))

# Learners for nuisance functions
y_learner <- lrn("regr.xgboost", eta = 0.1, nrounds = 35)
d_learner <- lrn("regr.xgboost", eta = 0.1, nrounds = 35)

# Storage for out-of-fold predictions
y_hat <- rep(NA_real_, nrow(dat))
d_hat <- rep(NA_real_, nrow(dat))
```

DML PLR By Hand

```
for (k in 1:K) {
  idx_test  <- which(fold_id == k)
  idx_train <- which(fold_id != k)

  # Train m(x): y ~ age on training folds
  y_task <- as_task_regr(dat[idx_train, c("income", "age")], target = "income")
  y_fit <- y_learner$clone(deep = TRUE)
  y_fit$train(y_task)

  # Train g(x): d ~ age on training folds
  d_task <- as_task_regr(dat[idx_train, c("d", "age")], target = "d")
  d_fit <- d_learner$clone(deep = TRUE)
  d_fit$train(d_task)

  # Predict on held-out fold
  y_hat[idx_test] <-
    y_fit$predict_newdata(dat[idx_test, "age", drop = FALSE])$response
  d_hat[idx_test] <-
    d_fit$predict_newdata(dat[idx_test, "age", drop = FALSE])$response
}
```

DML PLR by Hand

```r
# Cross-fitted residuals
y_resid <- dat$income - y_hat
d_resid <- dat$d - d_hat

# Second stage (PLR)
summary(lm(y_resid ~ d_resid))
```

```
Call:
lm(formula = y_resid ~ d_resid)

Residuals:
    Min      1Q  Median      3Q     Max
-3.9212 -0.6835 -0.0062  0.7177  3.8411

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) -0.03640    0.01451  -2.509   0.0121 *
d_resid      0.49533    0.03254  15.222   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.026 on 4998 degrees of freedom
Multiple R-squared:  0.04431,    Adjusted R-squared:  0.04412
```

A More Complex Example

A more complex dataset...

```
glimpse(df)
```

```
Rows: 5,000
Columns: 52
$ y   <dbl> 3.7264247, -2.4624067, 22.3863755, -13.1436464, -9.3678
$ d   <dbl> 2.4129374, -0.9491219, 8.3370590, -5.0850347, -3.296652
$ X1  <dbl> -0.56047565, -0.23017749, 1.55870831, 0.07050839, 0.129
$ X2  <dbl> -0.4941739, 1.1275935, -1.1469495, 1.4810186, 0.9161912
$ X3  <dbl> 2.37072522, -0.16681196, 0.92696140, -0.56815175, 0.225
$ X4  <dbl> -1.35384887, -0.57937725, -0.86104420, 0.97267834, 0.61
$ X5  <dbl> -0.83629674, -0.22057299, -2.10351477, -1.66780754, -1.
$ X6  <dbl> -0.79520751, -1.13511150, 0.58010122, 0.51804257, 0.207
$ X7  <dbl> -0.19360641, 0.25814728, -0.53831257, -1.17906281, 0.90
$ X8  <dbl> -0.88805283, 0.36026438, 0.42208992, -0.74323093, 0.180
$ X9  <dbl> 0.48252737, 0.72136466, -0.50779230, -0.06470875, 1.302
$ X10 <dbl> 0.97216119, -0.22347773, 0.14719695, -0.22580467, 0.583
$ X11 <dbl> 0.25958973, 0.91751072, -0.72231834, -0.80828402, -0.14
```

Single ML - not causal

```
set.seed(123)

# Use the mlr3 library to set up a machine learning workflow
# The glmnet.cv regression model uses Elastic Net regularization
# Here, we use lambda with lowest cross-validated error.
# Use lambda.1se for a more parsimonious model
lrner <- lrn("regr.cv_glmnet", s="lambda.min")

# Make a "training task" to target Y (outcome)
task <- as_task_regr(df, target='y')

# Train the model
lrner$train(task)

# Extract the coefficient for d directly
coef(lrner$model, s="lambda.min")['d',]
```

```
[1] 2.163535
```

DoubleML Package

```
pacman::p_load(DoubleML)
dmd <- double_ml_data_from_data_frame(df, y_col='y', d_cols='d')
dmd
```

```
================== DoubleMLData Object ==================


------------------ Data summary      ------------------
Outcome variable: y
Treatment variable(s): d
Covariates: X1, X2, X3, X4, X5, X6, X7, X8, X9, X10, X11, X12, X13,
Instrument(s):
Selection variable:
No. Observations: 5000
```

DoubleML Package

```
# Make a fresh instance of the learner object
ml_l_sim <- lrn("regr.cv_glmnet", s = "lambda.1se") # y learner
ml_m_sim <- lrn("regr.cv_glmnet", s = "lambda.min") # d learner
set.seed(1)
obj_dml_plr_sim <- DoubleMLPLR$new(dmd, ml_l = ml_l_sim, ml_m = ml_m_sim, n_folds =
obj_dml_plr_sim$fit()
```

```
INFO  [00:02:27.399] [mlr3] Applying learner 'regr.cv_glmnet' on task 'nuis_l' (ite
INFO  [00:02:27.480] [mlr3] Applying learner 'regr.cv_glmnet' on task 'nuis_l' (ite
INFO  [00:02:27.651] [mlr3] Applying learner 'regr.cv_glmnet' on task 'nuis_l' (ite
INFO  [00:02:27.714] [mlr3] Applying learner 'regr.cv_glmnet' on task 'nuis_l' (ite
INFO  [00:02:27.781] [mlr3] Applying learner 'regr.cv_glmnet' on task 'nuis_l' (ite
INFO  [00:02:27.878] [mlr3] Applying learner 'regr.cv_glmnet' on task 'nuis_m' (ite
INFO  [00:02:27.940] [mlr3] Applying learner 'regr.cv_glmnet' on task 'nuis_m' (ite
INFO  [00:02:28.002] [mlr3] Applying learner 'regr.cv_glmnet' on task 'nuis_m' (ite
INFO  [00:02:28.072] [mlr3] Applying learner 'regr.cv_glmnet' on task 'nuis_m' (ite
INFO  [00:02:28.135] [mlr3] Applying learner 'regr.cv_glmnet' on task 'nuis_m' (ite
```

```
obj_dml_plr_sim$summary()
```

```
Estimates and significance testing of the effect of target variables
  Estimate. Std. Error t value Pr(>|t|)
d   1.99892    0.01503     133   <2e-16 ***
---
```

DoubleML Package

```
# mtry = Number of variables randomly sampled as candidates at each split.
ml_l_rf = lrn("regr.ranger", max.depth = 5, mtry = 7, min.node.size = 5)
ml_m_rf = lrn("regr.ranger", max.depth = 5, mtry = 7, min.node.size = 5)
set.seed(1)
obj_dml_plr_rf <- DoubleMLPLR$new(dmd, ml_l = ml_l_rf, ml_m = ml_m_rf, n_folds = 5)
obj_dml_plr_rf$fit()
```

```
INFO  [00:02:28.370] [mlr3] Applying learner 'regr.ranger' on task 'nuis_l' (iter 1
INFO  [00:02:29.079] [mlr3] Applying learner 'regr.ranger' on task 'nuis_l' (iter 2
INFO  [00:02:29.763] [mlr3] Applying learner 'regr.ranger' on task 'nuis_l' (iter 3
INFO  [00:02:30.434] [mlr3] Applying learner 'regr.ranger' on task 'nuis_l' (iter 4
INFO  [00:02:31.106] [mlr3] Applying learner 'regr.ranger' on task 'nuis_l' (iter 5
INFO  [00:02:31.810] [mlr3] Applying learner 'regr.ranger' on task 'nuis_m' (iter 1
INFO  [00:02:32.508] [mlr3] Applying learner 'regr.ranger' on task 'nuis_m' (iter 2
INFO  [00:02:33.258] [mlr3] Applying learner 'regr.ranger' on task 'nuis_m' (iter 3
INFO  [00:02:33.967] [mlr3] Applying learner 'regr.ranger' on task 'nuis_m' (iter 4
INFO  [00:02:34.691] [mlr3] Applying learner 'regr.ranger' on task 'nuis_m' (iter 5
```

```
obj_dml_plr_rf$summary()
```

```
Estimates and significance testing of the effect of target variables
  Estimate. Std. Error t value Pr(>|t|)
d   2.88166    0.03427   84.08   <2e-16 ***
---
```